

IBM CIO "Software Developer" (Intern→FTE, 2027 Batch, India) — Stage-by-Stage Question Bank & Patterns (2024–2027)

TL;DR

Across all four stages, IBM CIO's Software Developer selection is a **pattern-consistent** process: two coding rounds dominated by LeetCode Easy–Medium DSA (arrays, strings, hashing, sliding window, DP, linked lists, matrices) plus SQL and pattern-printing; a Knockri asynchronous video round of ~6 STAR-style behavioral questions (1 min prep / 3 min answer) plus ~60 work-preference items; and combined in-person technical + HR interviews driven heavily by your resume, projects, DSA explanations, OOP/DBMS/OS fundamentals, and SQL.

The single most reliable predictor of success reported by candidates: **solve at least 2 full coding questions per round**, be able to explain your own code and projects deeply, and prepare 6 STAR+Learning stories including one on **responsible AI-tool use** (a new 2026–2027 theme).

These are **reported/pattern questions from past drives (2024–2027), useful for practice — not a leaked or guaranteed set**. IBM varies questions by role, location, campus, and year; recent reports confirm 2 coding questions in ~55–60 minutes with no MCQ in many drives, though some campuses still include 3–5 core-subject MCQs.

Key Findings

The coding rounds are the true filter. Stage 1 (virtual) and Stage 2 (in-person) both use HackerRank-style problems. Difficulty is LeetCode Easy–Medium, business/scenario-flavored (server loads, rate limiting, logins, IPs, circles). Round 2 frequently bundles a **SQL query (joins/group by)**, an **if-else/data-type logic** problem, and a **pattern-printing** problem. Solving 2 questions fully is generally enough to advance; solving fewer often means more technical grilling in the interview.

Knockri video assessment is competency-based, not technical. Current (2026–2027) reports describe **6 recorded behavioral questions, 1 minute to prepare / 3 minutes to answer, no re-attempts**, plus **~60 work-preference items on a 5-point scale**. IBM evaluates client focus, collaboration, adaptability, accountability, and innovation. A distinctive 2026–2027 addition: **questions about using AI tools and verifying AI-generated output**.

Interviews are resume-first and combined. For CIO Software Developer, the technical and HR rounds are often merged into one in-person session. Interviewers start with projects, then probe DSA (explain your Round-2 code), SQL, OOP, OS, DBMS, and (for non-CS branches like ECE/EEE/EIE) core-subject or "can you work as a software developer?" questions. Cloud/Docker/Kubernetes concepts recur.

Non-CS candidates get application-oriented questions. ECE/EEE/EIE candidates report being asked whether they can manage a software role, plus lighter/applied technical questions and project/ML questions

rather than deep CS theory.

Details by Stage

STAGE 1 & 2 — CODING ASSESSMENTS (Virtual + In-Person)

Format (reported, 2024–2027):

Platform: **HackerRank** (virtual round) and a supervised in-person lab (physical round).

Typically **2 coding questions in ~55 minutes** — NxtWave (ccbp.in) and PrepInsta's CodeKnack page both state the IBM coding assessment has "2 coding questions to be done in 55 minutes" and can be attempted in C, C++, Java, or Python (non-adaptive, no negative marking). Some drives vary: 3 questions, or 1 question in 45 min, or ~75 min offline with 3 tasks.

Languages: **C, C++, Java, Python.**

In-person Round 2 commonly = **(i) simple if-else/data-type logic, (ii) SQL query with joins/group by, (iii) pattern printing.** A GFG 2025 CIO Software Developer candidate (Coimbatore pooled drive) reported exactly this trio.

MCQs: Not always present. When included (some campus drives), **3–5 MCQs** cover **DBMS, Operating Systems, Computer Networks, OOP**, and occasionally aptitude (Venn diagrams, missing-number). One 2024 on-campus SE drive reported **15 MCQs + 2 coding**, with MCQs heavily weighted to **Operating Systems and Linux commands.**

Pass bar: Solve/near-solve **2 questions** (even with 1–2 failing test cases you may advance). Uniqueness/logic weighted over sheer count; **no plagiarism** (online round).

Below, coding questions are grouped by pattern. Each includes the reported problem, optimal approach, and complexity. Sources span GeeksforGeeks, LeetCode Discuss (via CodingKaro), Unstop, PrepInsta, techprogrammind, and Naukri Code360.

Arrays & Prefix Sums

Server load > twice average (2026, confirmed). Given `load[]` of server loads, return all 0-based indices where `load[i] > 2 × average`. *Approach:* one pass for total sum; compare `load[i]*n > 2*totalSum` using `long` to avoid float error. **$O(n)$ time, $O(1)$ space.**

Odd-Even Prefix Sum maximization (IBM CIO OA, 2024). Given an array, maximize sum by subtracting prefix sums at odd indices and adding at even indices, with rearrangement allowed. *Approach:* greedy sort/rearrangement + prefix-sum reasoning. **$O(n \log n)$.**

Prefix XOR → recover original array (PrepInsta bank). Given `pref[]` where `pref[i] = arr[0] ^ ... ^ arr[i]`, recover `arr`. *Approach:* `arr[0] = pref[0]; arr[i] = pref[i] ^ pref[i-1]`. **$O(n)$.**

EXOR sum / Maximum OR sum of subarrays (IBM CIO OA, 2022). Array subarray bitwise problems. *Approach:* bit-contribution counting. **$O(n)$ or $O(32n)$.**

Widest separation of differing features (Unstop 2026 bank). Given `features[]`, find $\max |i - j|$ where `features[i] ≠ features[j]`. *Approach:* two-pointer from both ends. **$O(n)$.**

Array rotation / Rotate array (IBM SD bank). *Approach:* reversal algorithm. **$O(n)$ time, $O(1)$ space.**

Minimum swaps to sort an array. *Approach:* cycle detection on index mapping. $O(n \log n)$.

Coin factory / digit-sum bucketing (Unstop 2026 bank). Place each ID in a box = sum of its digits; find the box with most coins over a range. *Approach:* digit-sum + count array of size 46. $O((hi-lo)-d)$.

Sorting / Two-Pointer / Dutch National Flag

Sort an array of 0s, 1s, and 2s (IBM SD, recurring). *Approach:* Dutch National Flag three-pointer. $O(n)$ time, $O(1)$ space.

Single element in a sorted array (IBM ASE, Jan 2025). *Approach:* binary search on parity index. $O(\log n)$.

Sliding Window

Frequent IPs in a time window (2026 NG track). Given IPs + sorted timestamps, find IPs appearing $> k$ times in any t -second window. *Approach:* two-pointer sliding window + hash map. $\sim O(n)$.

API Rate Limiting System (IBM OA, Mar 2025; recurs). Enforce per-domain request limits over time windows. *Approach:* per-key deque/rolling counter (token-bucket / 429 semantics). $O(1)$ amortized per request.

Maximize Decryption Value (IBM OA, LC Discuss #6910838, Jul 2025). Sliding-window maximization variant (candidate referenced LC 239). Hard-tagged.

Hashing / Strings

Repeated DNA Sequences (IBM ASE On-Campus, Jan 2025; ~LC 187). Return all 10-letter substrings occurring > 1 time. *Approach:* hash set of fixed-length substrings / rolling hash. $O(n)$.

DNA/RNA decode (IBM ASE, Nov 2024). Decode a binary/encoded string into `"TYPE: SEQUENCE"` (DNA or RNA). *Approach:* fixed-width chunk parse + mapping. $O(n)$.

Top K Frequent Words (IBM CIC 2025). Return k most frequent words, ties broken lexicographically. *Approach:* hash-map frequency + sort/heap by `(-count, word)`. $O(n \log n)$ or $O(n \log k)$.

Most frequent non-banned word. *Approach:* hash-map frequency, skip banned set. $O(n)$.

Phone number in words $\rightarrow$ digits (IBM CIC 2025). Convert `"six four eight three"` $\rightarrow$ `"6483"`, handling `"double"`/`"triple"` multipliers. *Approach:* word $\rightarrow$ digit map + multiplier state. $O(n)$.

Proper string compression (PrepInsta bank). Compress `"a3b5c2a2"` $\rightarrow$ aggregate counts per char in alphabetical order. *Approach:* frequency array of 26. $O(n)$.

Length of last word with multiple spaces / trailing chars (IBM SE 2024). *Approach:* trim + scan from end. $O(n)$.

Consecutive-error resolver (IBM CodeKnack CIC 2024). Count times an error occurs $3\times$ consecutively per process and must be resolved. *Approach:* per-key running counter + reset. $O(n)$.

Common characters in two strings; Reverse a string; Longest palindrome. Standard string patterns; hashing/two-pointer. $O(n)-O(n^2)$.

Linked Lists

Delete nodes in a value range (IBM CIO OA, 2024). Given head, return list without nodes whose values fall in `[lo, hi]`. *Approach:* dummy-node traversal with pointer relink. $O(n)$.

Add two numbers as linked lists (IBM ASE 2025). Digits in reverse order. *Approach:* simultaneous traversal + carry. $O(\max(m,n))$.

Detect loop / circular linked list; Reverse a linked list. *Approach:* Floyd's cycle detection; iterative reversal. $O(n)$ time, $O(1)$ space.

Matrices / 2-D DP

Count square sub-islands / count square submatrices of 1s (Unstop 2026 bank). *Approach:* DP $dp[i][j] = 1 + \min(\text{top}, \text{left}, \text{diag})$ when cell=1; sum all. $O(n \cdot m)$.

Circle relationships (IBM CodeKnack CIC 2024). Given $[x1 \ y1 \ r1 \ x2 \ y2 \ r2]$, classify two circles as concentric/touching/intersecting/disjoint. *Approach:* compare center distance vs radii sums/differences. $O(1)$ per pair.

Chessboard key movement (IBM App Developer, on-campus). 8×8 board, apply move instructions ("u6" = up), print final position or "Invalid input" if out of bounds. *Approach:* bounded simulation. $O(k)$.

Dynamic Programming / Greedy / Game Theory

Optimal stone-picking game (Aryan/Ankit, Unstop 2026 bank). Two players pick from either end; both optimal; decide winner. *Approach:* interval DP $f(i,j) = \max(v[i] + f(i+1,j), v[j] + f(i,j-1))$. $O(n^2)$.

Greedy + Max Heap (GFG 2025 CIO Software Developer, Round 1). Round-1 virtual = 2 questions on Greedy and Max Heap, moderate difficulty. *Approach:* priority queue. $O(n \log n)$.

Coin Change; Minimum days to complete work; Minimum elements (min of stream). Classic DP/greedy/min-tracking. $O(n \cdot \text{amount})$ / $O(n)$ / $O(1)$ per op with min-var.

Maximum Distinct Split Sum (IBM OA, LC Discuss #6910838). Partition into max distinct segments. *Approach:* backtracking (small n) / partition DP. Hard-tagged.

Longest Bitonic Subarray (IBM OA, Mar 2025). Longest subarray that increases then decreases. Example $[10, 8, 9, 15, 12, 6, 7] \rightarrow 5$. *Approach:* two-pass inc[]/dec[] arrays. $O(n)$.

Number Theory / Math / Bit Manipulation

HCF/GCD without recursion (IBM classic). *Approach:* iterative Euclid / subtraction. $O(\log \min)$.

LCM of three numbers; Decimal $\rightarrow$ binary; Fibonacci triangle (PreplInsta/IBM bank). Standard math loops.

Swap MSB and LSB of an integer; detect Little vs Big endian (IBM SE tech round). Bit manipulation.

IPv4 $\rightarrow$ IPv6 conversion (IBM SE on-campus 2024). String/number parsing.

FizzBuzz (IBM CIC 2025). *Approach:* modulo checks. $O(n)$.

Minimum total cost to select all items (2026). Pick cheapest cost per item id; -1 if any missing. *Approach:* min-array per id. $O(n)$.

Nth Fibonacci; Two-sum to target; Largest of three integers. Foundational.

Grid / Simulation / Greedy Path

Earliest time to reach target on a grid (2026 NG track). Move per direction string or stay; return earliest arrival or -1. *Approach:* cumulative direction counts / greedy feasibility. $O(n)$.

Login replacement counter (2026 AI-Engineer track). Count replacements when a login_id hits 3 consecutive errors (then resets). *Approach:* hash-map running counter. $O(n)$.

STAGE 3 — KNOCKRI ASYNCHRONOUS VIDEO ASSESSMENT

What it is: IBM's asynchronous, AI-scored competency interview run on **Knockri** — a structured-interview platform headquartered at 150 King St W, Toronto, co-founded in 2016 by CEO Jahanzaib Ansari, COO Maaz Rana, and CTO Faisal Ahmed; per Dealroom.co it "serves major corporations and government bodies, including IBM, Deloitte, SAP, and the Government of Canada." Per Knockri's official Candidate FAQ (knockri.com/candidate-faq), "Our AI turns your spoken responses into text and evaluates them against skills and competencies the role requires... final decisions are always made by people, not AI." IBM's own materials confirm candidates may face video, coding, English-language, or work-behaviour assessments depending on role. [ibm](#)

↗

Format (2026–2027 candidate reports):

~6 recorded behavioral questions. Per PracHub's "IBM Competency Assessment 2027" guide: "In a current August 2026 candidate report, the assessment contained six video questions plus about 60 work-preference items." (Some 2026 accounts say 5–8 — Leland's IBM Consulting Interview Guide describes "5-8 behavioral and situational IBM interview questions.")

1 minute to prepare, 3 minutes to answer per question — confirmed by MConsultingPrep: "Candidates need approximately 10 minutes for this assessment, with 1 minute for preparation after receiving each question and 3 minutes to answer each question." (One conflicting 2026 marketing-blog account — Leland — claims ~30 seconds to prepare, 2–3 minutes to record, and "1-2 retry attempts per question"; the more reliable Reddit-sourced reports say **no re-attempts**, so prepare accordingly.) [PracHub](#)

↗

~60 work-preference items on a 5-point scale, often forced-choice between two statements ("select the one you agree with slightly more"). [PracHub](#)

↗

Whole video portion ~10 minutes. Unlimited practice questions before the scored section.

Competencies IBM evaluates: Per an IBM-focused 2026 guide (linkjob.ai), interviewers "evaluate whether you align with the company's core competencies (including client focus, innovation, collaboration, accountability, and adaptability)" — plus trust. PracHub maps the 6 questions to: collaboration; conflict/feedback; ownership; learning from failure; ambiguity/priorities; and **responsible AI use**.

Reported/sample prompts (STAR-style; practice, not guaranteed):

"Tell me about a time you overcame a challenge." (Knockri official FAQ sample) [Knockri](#)

↗

"Describe a time you helped someone in need." (Knockri official FAQ sample) [Knockri](#)

↗

"Tell me about a time you had to work in a team under pressure." [Leland](#)

↗

"Tell me about a time you led a cross-functional team." (2026 account)

"How would you handle a client who is dissatisfied with your solution?" (2026 account) [Leland](#)

↗

"Tell me about a time you had to adapt to a significant change."

"Describe a time you made a mistake" / learned from failure.

"How do you handle tight deadlines?"

AI-tools questions (2026–2027, distinctive): Per PracHub, "Current candidates specifically mention questions about using AI tools, evaluating their capabilities, and verifying generated output." Prepare one genuine example emphasizing your verification method.

Motivation items ("why this role / why IBM / how you add value").

How to answer: Use **STAR+L (Situation, Task, Action, Result, Learning)**. Budget: ~15–25s situation, 10–15s task, 60–90s action (your decisions + reasoning), 20–30s result (measurable), 15–25s learning. Build a **six-story evidence bank** (team disagreement, tough deadline, a mistake, an ambiguous problem, an unprompted improvement, responsible AI use). For work-preference items: answer **honestly and consistently**; pick the statement that fits slightly better; don't try to reverse-engineer an "ideal" persona.

STAGE 4 — IN-PERSON TECHNICAL & HR INTERVIEW

For CIO Software Developer, technical + HR are frequently **combined into one in-person round** (sometimes two interviewers). It is **resume- and project-first**. Results are often announced on-campus the same/next day.

Opening / resume-based (very common):

Walk me through your resume / self-introduction.

Explain your projects; what was your specific contribution; challenges faced and how you overcame them.

[geeksforgeeks](#)

↗

Why did you choose your degree? (and any education gap — keep ≤1 year, explain honestly) [geeksforgeeks](#)

↗

Rate yourself in Java/Python/SQL. [GeeksforGeeks](#)

↗

Latest technologies / industry trends you follow.

DSA & coding (they often re-open your Round-2 code):

Explain the logic of your pattern-printing / coding solution from the offline round; **optimize it**; solve a problem you couldn't finish earlier. [geeksforgeeks](#)

↗

[GeeksforGeeks](#)

↗

Create a linked list node (in C/your language); detect a cycle in a linked list.

Explain searching/sorting algorithms; types of time complexity; heap sort dry run. [codingkaro](#)

↗

Bit manipulation: swap MSB/LSB; little vs big endian.

Print the alphabet 'N' using any language (scenario coding). [geeksforgeeks](#)

↗

OOP (near-guaranteed):

Four pillars of OOP (encapsulation, inheritance, polymorphism, abstraction). [naukri](#)

↗

[Soshalcare](#)

↗

Types of inheritance; important keywords; function overloading vs overriding; abstract class vs interface; constructor overloading.

DBMS / SQL (near-guaranteed):

Write a query using GROUP BY; find the second-highest salary and optimize it; SQL with joins; order of execution for multiple joins; scenario-based DB question. [geeksforgeeks + 3](#)

↗

Normalization; ACID properties; primary key; SQL vs NoSQL; denormalization reasons; concurrency in DB updates. [Soshalcare](#)

↗

[codingkaro](#)

↗

Operating Systems / Linux:

Process vs thread; multithreading and how it's handled; semaphore vs mutex; deadlock; Banker's algorithm; CPU scheduling; paging/virtual memory/segmentation; full form of LINUX + basic commands; software interrupts; dangling pointer. [codingkaro + 2](#)

↗

Computer Networks:

OSI model layers; TCP three-way handshake; TCP vs UDP; what happens when you enter [www.ibm.com](#) (web request flow); port forwarding. [codingkaro](#)

↗

Cloud / DevOps (recurs at IBM):

Docker vs virtualization; purpose of Docker / Docker images; Kubernetes vs Docker / orchestration; VM resource allocation; private vs hybrid cloud; microservices vs monolithic; cloud certifications (mention if on resume to steer the conversation). [geeksforgeeks + 2](#)

↗

Scenario/design (lightweight):

Implement cart-checkout discounts with credit-card validation and order amount. [geeksforgeeks](#)

↗

Factors in designing a website for a grocery store; how web pages are rendered. [geeksforgeeks](#)

↗

For non-CS branches (ECE/EEE/EIE): Expect application-oriented rather than deep-CS questions, plus: "Being an ECE student, can you manage/work as a software developer?"; project-based ML questions if your projects are ML/AI (gradient descent, loss function, F1 score vs accuracy, overfitting/underfitting,

epochs/batch size/learning rate); occasionally core-electronics (CMOS, K-map, flip-flops) if relevant to your resume. [geeksforgeeks + 2](#)

↗

HR / Managerial questions (typical):

Tell me about yourself.

Why IBM? What do you know about IBM / its technologies? [Glassdoor](#)

↗

Why are you a good fit for the Software Developer role? [Glassdoor](#)

↗

Strengths and weaknesses; how do you handle failure/stress/tight deadlines. [Soshalcare](#)

↗

Are you willing to relocate (anywhere in India)? Willing to work overtime/nights/weekends? [InterviewBit](#)

↗

[Hitbullseye](#)

↗

Where do you see yourself in 5 years? Long-term goals. [Glassdoor](#)

↗

Will you adapt to a project outside your expertise / can you work independently? [GeeksforGeeks](#)

↗

If you had multiple offers, why choose IBM? What makes you distinct? [codingkaro](#)

↗

Situational: resolving conflict, prioritizing competing deadlines, handling a difficult client/teammate.

Recommendations

Stage 1-2 (Coding) — do this first and most:

Grind **LeetCode Easy-Medium** with priority on **arrays, strings, hashing, sliding window, linked lists, matrices/2-D DP, and greedy** — the exact clusters above. Target ~150-250 problems.

Drill **pattern printing** (pyramids, diamonds with conditions), **SQL joins/group by/second-highest-salary**, and **if-else/data-type conversion** — the Round-2 trifecta.

Practice under a **timer (2 questions/55-60 min)**; aim to fully solve 2 with clean, commented code and edge cases. Benchmark: if you can't consistently clear 2 mediums in 60 min, keep drilling before the drive.

Skim **OS + Linux commands, DBMS, CN, OOP** for MCQs (only some campuses include them).

Stage 3 (Knockri): 5. Pre-build **6 STAR+L stories**, including one **responsible-AI-use** story (how you verified AI output). Record yourself with **1 min prep / 3 min answer** and **assume no retakes**. 6. Do the platform's practice questions; test camera/mic/connection 24h ahead; answer work-preference items **honestly and consistently**. [Knockri](#)

↗

[Leland](#)

↗

Stage 4 (Interview): 7. Know **every line of your resume and projects** — this is where most questions originate. Be ready to optimize your own Round-2 code. 8. Prepare crisp answers on **OOP, SQL (write live queries), OS, CN, and Docker/Kubernetes/cloud**. Non-CS candidates: prepare a confident "yes, I can be a

software developer" narrative + project depth. 9. Rehearse HR staples (**why IBM, relocation = yes, 5-year goal, strengths/weaknesses**) in STAR form.

Thresholds that change the plan:

If your campus variant includes **MCQs or an English/GD round** (some pooled/CIC drives do), add core-subject MCQ practice and a short GD prep.

If you receive the **Knockri link before/without a coding OA**, treat it as a required gate regardless of order — don't assume it means you passed coding.

If you solve **fewer than 2** coding questions, over-prepare interview technical depth: interviewers grill low-scorers harder.

Caveats

These are reported and pattern-based questions compiled from public candidate experiences (2022–2027) across GeeksforGeeks, LeetCode Discuss/CodingKaro, Unstop, PrepInsta, techprogrammind, Naukri Code360, Glassdoor, PracHub, MConsultingPrep, and Knockri/IBM official pages. They are for practice; **IBM does not publish a fixed question set and varies questions by role, location, campus, batch, and year.** Treat nothing here as leaked or guaranteed.

Knockri format numbers (6 questions, 1 min/3 min, ~60 preference items, no retakes) are candidate-reported, not officially published by IBM, and one source (Leland) conflicts on timing/retakes. Your invitation and on-screen instructions are the final authority.

Some coding "problem banks" (PrepInsta, Unstop, Scribd, techprogrammind) reconstruct problems from memory; exact wording/constraints in your test may differ. A few aggregator sources promote AI interview-cheating tools — the problem statements corroborate independent reports, but ignore the cheating pitch (IBM's 2026–2027 process specifically probes responsible AI use and proctors OAs in some configs).

The CIO Software Developer role is **intern→FTE**; process specifics (rounds, GD presence, MCQs, whether HR is separate) shift between pooled off-campus drives, CIC/CodeKnack campus drives, and ISDL/ASE variants.

The coding-assessment format overlaps heavily across these, which is why ASE/SE data is included as directly relevant.